



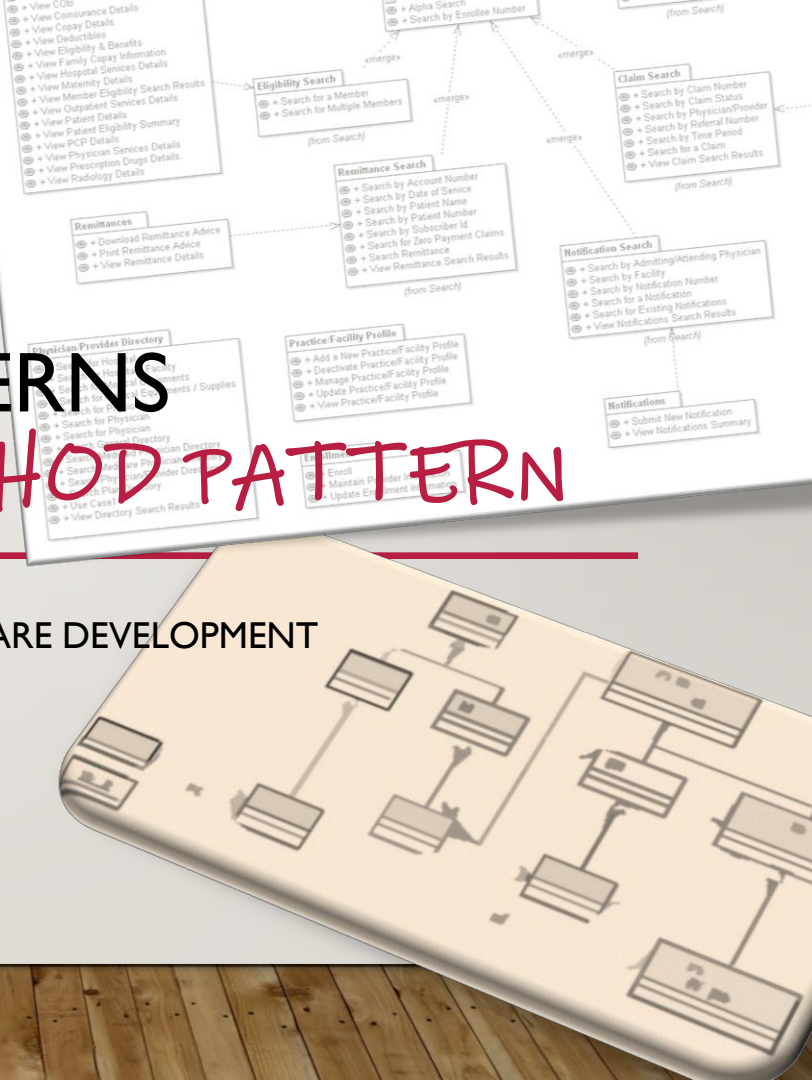
THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tamaki Makaurau
NEW ZEALAND

OO DESIGN PATTERNS

-TEMPLATE METHOD PATTERN

COMPSCI 230 OBJECT-ORIENTED SOFTWARE DEVELOPMENT

PARAMVIR SINGH

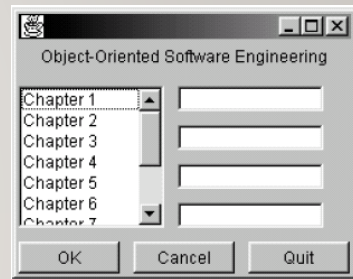
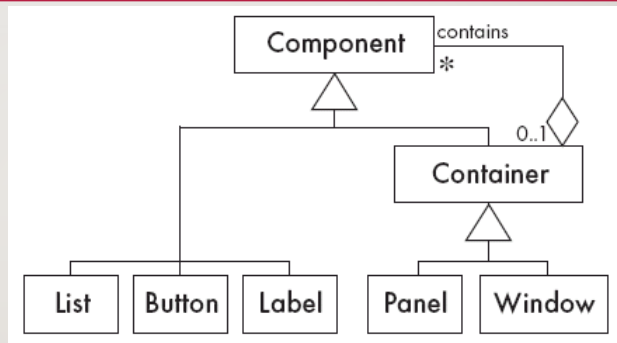


AGENDA

- Q/A Discussion
 - Composite Pattern lecture exercises
- Pattern of the day
 - Template Method Pattern

Q/A DISCUSSION – COMPOSITE PATTERN

- **Composite pattern in Java API?**



- **Design objectives addressed by composite pattern?**

- Higher flexibility – more `FileSystemItem` subclasses may be added without affecting other classes
- Higher robustness – use of type-safety via generic list (`List<FileSystemItem>`), use of runtime polymorphism in `printFileHierarchy`, use of protected field in superclass

- **SOLID principles followed by composite pattern?**

- Follows OCP, LSP and DIP

RECALLING ACCESS MODIFIERS

RECALL PART-I

	default	private	protected	public
Same Class	Yes	Yes	Yes	Yes
Same package subclass	Yes	No	Yes	Yes
Same package non-subclass	Yes	No	Yes	Yes
Different package subclass	No	No	Yes	Yes
Different package non-subclass	No	No	No	Yes

A QUICK BUILD UP EXERCISE!

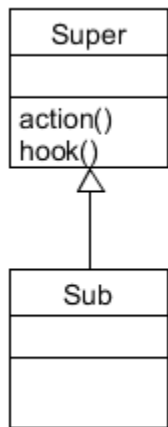
RECALL PART-I

```
class Super {  
    public void action( ) {  
        System.out.print( "0" );  
    }  
}  
  
class Sub extends Super {  
    public void action( ) {  
        System.out.print( "1" );  
    }  
}
```

- What is printed by the following:

```
Super s1 = new Super( );  
Super s2 = new Sub( );  
Sub s3 = new Sub( );  
  
s1.action( ); 0  
s2.action( ); 1  
s3.action( ); 1
```

INVERSION OF CONTROL



```

class Super {
    public void action() {
        hook();
    }

    protected void hook() {
        System.out.println("Super's hook");
    }
}

class Sub extends Super() {
    protected void hook() {
        System.out.println("Sub's hook");
    }
}
  
```

- What is printed by the following:

```

Super s1 = new Sub();
s1.action();
  
```

Sub's hook

- Super's action() method is a **Template method**
- What is a Template method useful for?

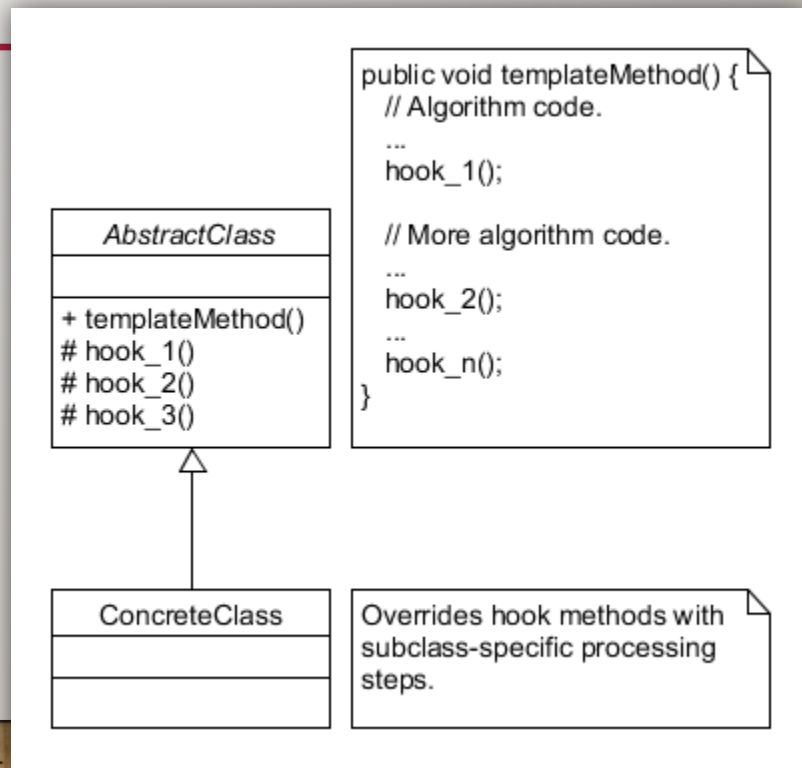
TEMPLATE METHOD PATTERN

- **Context:**
 - An operation (or underlying algorithm) comprises multiple steps, where each step is implemented by a method.
- **Problem:**
 - How can you create multiple implementations of one of the steps, while keeping the other steps as same?
- **Forces:**
 - The algorithm implements a service that the encapsulating class is responsible for
 - The service should be provided either by the class alone or its inheritance hierarchy
- **Reference:**
 - E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, October 1994.

TEMPLATE METHOD PATTERN

Solution:

- **Template Method** is a technique that defines the **skeleton of an algorithm** in a superclass method
- Template Method allows subclasses to **redefine certain steps** of an algorithm without changing the algorithm's structure



TEMPLATE METHOD PATTERN *EXAMPLE-1*

To Eclipse >>

- MobileGame's play() template method

TEMPLATE METHOD PATTERN *EXAMPLE-2*

Shapes application:

- Implement the `DynamicRectangleShape` class using Template method pattern.

```
class Shape {
    ...
    // Type to represent bounce surfaces.
    protected enum Surface { North, East, South, West,
        NorthEast, SouthEast, SouthWest, NorthWest };

    public final void move( int width, int height ) {
        Modified to become a Template method that calls hook
        method shapeHasMoved( ) each time move( ) executes.
    }

    protected void shapeHasMoved( BounceSurface surface ) { }
    // Optional hook method with null parameter if shape has
    // moved but not bounced; non-null parameter otherwise
    // indicating surface bounced off.
}
```

```
class DynamicRectangleShape extends Shape {
    protected Color _color;
    protected BounceSurface _bouncedOff;

    protected void shapeHasMoved( BounceSurface surface ) {
        _bouncedOff = surface
    }

    public void paint( Painter p ) {
        Use Painter's drawRect( ) or fillRect( ) method
        depending on _bouncedOff.
    }
}
```

THE TEMPLATE METHOD PATTERN

APPLICABILITY

- You want to implement invariant parts of an algorithm **once** and leave it up to subclasses to implement the variable behaviour
- Factor and localise behaviour among subclasses in a common class to **avoid code duplication**
- **Control subclass extension.** A template method that calls hook methods at specific points facilitates extension only at these points

IMPLEMENTATION

- Give hook methods **protected** access
 - They're not intended to be called by clients
- Define **mandatory** hook (slot) methods as **abstract** in the superclass
 - They're steps that subclasses **must** implement
- Implement **optional** hook (also known as primitive) methods in the superclass
 - They're steps that subclasses **may** provide behaviour for
- Define template methods as **final**
 - Subclasses shouldn't make **arbitrary** changes to the behaviour of a template method

SUGGESTED TASKS

- Complete the implementation of **MobileGame** example code
- Which **design objectives** are helped by the **design pattern** studied in today's class?
- Can you see **some instances of SOLID principles** being followed (or not followed) – in the examples of this design pattern?
- Are there instances of template pattern implemented in the Java API?

REVIEW

- **Learnt something about:**
 - **Design patterns**
 - Design patterns are proven, generalisable and reusable design techniques for common design problems
 - Design patterns have stated benefits and are widely used in modern object-oriented software development
 - **Template Method Pattern**
 - involves creating a special **template method** in a superclass, usually an abstract superclass,
 - defines the skeleton of an operation in terms of a number of high-level steps.
 - These steps are themselves implemented by additional helper **methods** in the same class as the **template method**
 - The **hook methods (abstract steps)** can be implemented in a variety of ways in concrete subclasses of the class defining the template method.

REFERENCES

➤ [Books]

- Chapter 6 – Design Patterns (**Canvas**), Object-Oriented Software Engineering: Practical Software Development using UML and Java by Timothy Lethbridge and Robert Laganieri
- Design Patterns: Elements of Reusable Object-Oriented Software by John Vlissides, Ralph Johnson, Richard Helm, Erich Gamma
- Agile Software Development, Principles, Patterns, and Practices by Robert C Martin

➤ [Media]

- Interesting article on application of Template Pattern on Sorting
 - https://cnx.org/contents/QCsgrcAf@37.6:VirdRf_z@5/Design-Patterns-for-Sorting
- A JavaWorld introductory article on design patterns
 - <http://www.javaworld.com/javaworld/jw-10-2001/jw-1012-designpatterns.html>

➤ [See more on Canvas Lecture page]